Minimum Spanning Trees

Intro to Minimum Spanning Trees:

Spanning Tree:

- A subgraph of a connected, undirected graph that includes all the vertices of the original graph, connected by a subset of the edges, such that there are no cycles

Minimum Spanning Tree (MST):

- A spanning tree of a connected, weighted, undirected graph with the smallest possible total sum of edge weights (the cheapest way to connect every vertex)

MST Properties:

- **Edges:** They have $|V| - 1$ edges
- **Non-Uniqueness:** A graph can have multiple different Spanning Trees. And if several edges have the same weight, then a graph can also have multiple *Minimum Spanning Trees*
- **Cycle Property:** Given any cycle within a graph, the largest edge will never be part of the Minimum Spanning Tree (MSTs can't have cycles)
- **Cheapest Connector:** If you divide the graph's vertices into two separate groups. the cheapest edge that connects a vertex in group A to a vertex in group B, must be part of the Minimum Spanning Tree

Note on Adjacency Matrices:

- When representing weighted graphs in an adjacency matrix, an entry of $A(i, j) = 0$ means that the edge doesn't exist. Which is different to having an edge of weight 0
- Therefore, we treat non-existent edges as if they have a weight of ∞ so the greedy algorithm never picks them

Greedy Algorithms:

- An algorithm that makes the best possible choice at every individual step, hoping that these local "best" choices would lead to the overall global "best" solution (no long-term thinking)
- **MST Algorithms:** At every step, we always choose the cheapest edge available that doesn't break any rules (like creating a cycle)
- Both Kruskal's and Prim's algorithm are greedy algorithms

Generic MST Algorithm Framework

Both Kruskal's and Prim's are just specific implementations of this broader mathematical theory

- **Cut:** A cut $(S, V - S)$ is a partition of the vertices into two completely separate groups
- **Crossing:** An edge crosses the cut of one endpoint is in set S, and the other is in set $V - S$
- **Respecting a Set:** A cut "respects" a set of chosen edges A if absolutely no edge from A cross the cut
- **Light Edge:** The edge with the absolute smallest weight that crosses a cut
- **Safe Edge:** If A is a valid subset of an MST, an edge (u, v) is "safe" if adding it to A keeps it a valid subset of an MST
    - ***Theorem:*** *If a cut respects A, then Light Edge crossing that cut is mathematically guaranteed to be a Safe Edge*

Kruskal's Algorithm

- An "Edge-First" approach to finding a Minimum Spanning Tree
- Sorts the list of edges from cheapest to most expensive, and attempts to add them to the current solution in that order (Greedy Approach)

Method:

1. **Sort Edges:** List all the edges in ascending order of their weight
2. **Setup:** Start with a **Forest** where every vertex is its own tiny, isolated tree (no edges yet)
3. **Greedy Loop:** Iterate through the sorted edges. For each edge (u, v):
    - **Check for Cycles:** If u and v are already in the same tree, **discard** the edge (u, v) to avoid creating a cycle
    - **Union:** If u and v are in different trees, **add** the edge to your MST and merge the two trees into one
4. **Finish:** Stop once you have added exactly $V - 1$ edges or have checked every edge

---

The Union-Find Data Structure (Disjoint Subsets):

Kruskal's algorithm requires us to dynamically partition our vertices into a collection of **disjoint (completely separate) subsets**

Three Core Operations

- `makeset(x)` **:** Creates a brand new, one-element set containing just x (Initialising an element takes $O(1)$ time)
- `find(x)` **:** Returns the **Representative** of the subset containing the element x (Checking an element takes $O(1)$ time)
  - If `find(u) == find(v)` , then they are in the same set (Joining u and v would create a cycle)
- `union(x, y)` **:** Joins the two completely separate subsets containing x and y into one single, merged set

Representatives & Arrays:

Representative:

- To check if two vertices are in the same subset, the data structure assigns one specific element to be the "Representative" for the entire group. This is usually the smallest integer in the set

The Representative Array:

- Each *Index* of the array is the vertex itself, and the *Value* stored at that index is its Representative

---

Example: Union-Find Data Structure

Imagine a graph with 6 vertices: $\{1, 2, 3, 4, 5, 6\}$

1. Run `makeset(x)` on all 6 vertices, giving us 6 separate sets:

$$\{1\}, \{2\}, \{3\}, \{4\}, \{5\}, \{6\}$$

2. Suppose the algorithm chooses to join edges $(1, 4)$ and $(5, 2)$. So we run `union(1, 4)` and `union(5, 2)` to get:

$$\{1, 4\}, \{5, 2\}, \{3\}, \{6\}$$

3. Then the algorithm wants to join edges $(4, 5)$ and $(3, 6)$. So we run `union(4, 5)` and `union(3, 6)` to get:

$$\{1, 4, 5, 2\}, \{3, 6\}$$

4. Now we are left with two final disjoint subsets: $\{1, 4, 5, 2\}, \{3, 6\}$

At this point, because 1 is the smallest number in the first set, and 3 is the smallest in the second set, they act as Representatives of their respective subsets.

Representative Array:

Index (Elements)	1	2	3	4	5	6
Array Value (Representative)	1	1	3	1	1	3

Real-Life Implementation of Union-Find:

Array Limitations:

- If the vertices are scattered numbers (e.g. 10, 55, 99) then a standard array would need 100 memory spaces just to store three values (wasted space)
- If vertices are strings (e.g. "A", "B", "C"), then an array can't use them as indices

The Solution (Hash Maps):

Modern Union-Find implementations replace the rigid array with a **Hash Map** (a `dictionary` in Python)

- A dictionary maps a **Key** (the name/label of the vertex) to a **Value** (its Representative)
- Dictionaries don't rely on sequential memory, and the "Index" can be any data type

---

Code Implementation: Kruskal's Algorithm (Tested in VS Code)

```
class Graph:
    def __init__(self, vertices, edges):
        # The list of all vertices in the graph
        self.vertices = vertices
        # The list of edges, where each edge is a tuple in the format:
(weight, u, v)
        self.edges = edges

# The Hash Map (dictionary) used to map a vertex to its Representative
# Declared globally so our isolated functions can access it without passing
it as a parameter
rep = {}

# Creates a one element set x
def makeset(x):
    rep[x] = x

# Returns the representative of the subset containing x
def find(x):
    # Base Case: If x is its own representative, we have found the root of
the tree. Stop and return it.
```

```
    if rep[x] == x:
        return x

    # Recursive Step: If x is not the root, dig deeper and update x's
representative (and all of its parents' reps) to be the root
    rep[x] = find(rep[x])
    return rep[x]

def union(x, y):
    # Constructs the union of the disjoint subsets containing x and y
    root_x = find(x)
    root_y = find(y)

    # Arbitrarily make root_x the representative for both sets (making the
newly combined set)
    if root_x != root_y:
        rep[root_y] = root_x

def kruskal_mst(G):
    # Clear the global dictionary in case we run the algorithm multiple
times
    rep.clear()

    # The list of chosen edges that form the Minimum Spanning Tree
    mst = []

    # Setup: Start with a Forest where every vertex is its own isolated tree
    for v in G.vertices:
        makeset(v)
    # Now rep = {"A":"A", "B":"B" ...}

    # Sort Edges: List all the edges in ascending order of their weight
    # sorted() Works without extra parameters because the weights are the
first elements in each edge's tuple
    sorted_edges = sorted(G.edges)

    # Greedy Loop: Iterate through the sorted edges
    for edge in sorted_edges:
        weight, u, v = edge

        # If the vertices have different representatives (to avoid cycles):
        if find(u) != find(v):
            # Add the edge to the MST
            mst.append(edge)
            # Merge the two trees into one
            union(u, v)

            # Pruning: Stop early if we have the required V - 1 edges
            if len(mst) == len(G.vertices) - 1:
                break
```

```python
    return mst

if __name__ == "__main__":
    # A dummy list of 6 vertices
    vertices = [1, 2, 3, 4, 5, 6]

    # A dummy list of edges in the format (weight, u, v)
    edges = [
        (1, 1, 4), (2, 5, 2), (3, 4, 5), (4, 3, 6),
        (5, 1, 2), (6, 3, 4)
    ]

    # Instantiate the Graph and run Kruskal's algorithm
    G = Graph(vertices, edges)
    mst_result = kruskal_mst(G)

    print("Edges in the Minimum Spanning Tree:")
    for weight, u, v in mst_result:
        print(f"Edge ({u}, {v}) with weight {weight}")
```

Path Compression

Diagram:

```
BEFORE Path Compression:       AFTER Path Compression:
(A long, inefficient chain)    (A flattened, efficient tree)

     [A] (Root)                     [A] (Root)
      |                            / | \
     [B]                         [B][C][D]
      |
     [C]
      |
     [D] <--- find(D)
```

Going down the recursive stack:

- When `find(D)` is called, it sees that it isn't the root and calls `find(C)`, which calls `find(B)`, which calls `find(A)`. We hit the **Base Case** at node A (the root)

Going back up the recursive stack:

- **Return the root:** Node A returns itself to the paused `find(B)` job
- This triggers the `rep[B] = A` assignment so that Node B points to Node A

- Then `find(B)` returns A to the paused `find(C)` job, executing `find(C) = A` so that Node C points to Node A, and returns A to the original `find(D)` job, executing `rep[D] = A`

Flattening Effect:

- Because the assignment occurs at *every single step* as we go back up the recursion stack, each node between D and A points straight to the root of the subgraph (A)

Optimisation:

- This deep, recursive climb only has to happen once per long branch, because now all the vertices in the subgraph (between A and D) are directly connected to the root
- Therefore, any future `find()` calls on B, C or D will hit the base case immediately, running in $O(1)$ time

---

Example: Path Compression for `find()`

Initial State:

(Assume that A, B, C, D are a subgraph of a larger graph, so the pruning step doesn't trigger)

- **Vertices:** $\{A, B, C, D\}$
- **Edges:** $[(1, C, D), (2, B, C), (3, A, B), (4, D, A) \dots]$
- $\text{mst} = \emptyset$
- $\text{rep} = \{A : A,\ B : B,\ C : C,\ D : D\}$

1. Check Edge (1, C, D)

- The Roots differ (C and D), so we **Add to MST**, calling `union(C, D)` joins the two subgraphs (the isolated nodes C and D) by having them share a root (rep) with `rep[D] = C`
- $\text{rep} = \{A : A,\ B : B,\ C : C,\ \textbf{D: C}\}$

2. Check Edge (2, B, C)

- Roots differ (B and C). **Add to MST.** `union(B, C)` sets `rep[C] = B`
- $\text{rep} = \{A : A, B : B, \textbf{C: B}, D : C\}$ *(A chain is forming: $D \to C \to B$)*

3. Check Edge (3, A, B)

- Roots differ (A and B). **Add to MST.** `union(A, B)` sets `rep[B] = A`
- $\text{rep} = \{A : A, \textbf{B: A}, C : B, D : C\}$ *(The long, inefficient chain is complete: $D \to C \to B \to A$)*

4. Check Edge (4, D, A) - Path Compression Triggers here!

- `find(A)` immediately returns the root A
- `find(D)` climbs the long chain, tracing $D \to C \to B \to A$ to get to the root A
- **Path Compression:** executes as the recursive stack unwinds, updating the roots of all traversed nodes to point directly to the root `rep[C] = A` and `rep[D] = A`
- Both roots are now A. Cycle Detected. **Discard Edge**
- $\text{rep} = \{A : A, B : A, \mathbf{C: A}, \mathbf{D: A}\}$ *(The tree is now completely flattened!)*
- **Optimisation:** Any future edges checked involving B, C, or D will now hit the base case immediately and run in instantaneous $O(1)$ time

Diagram: (Again)

```
BEFORE Path Compression:       AFTER Path Compression:
(A long, inefficient chain)    (A flattened, efficient tree)

     [A] (Root)                     [A] (Root)
      |                            / | \
     [B]                         [B][C][D]
      |
     [C]
      |
     [D] <--- find(D)
```

Kruskal's Algorithm: Complexity Analysis

Time Complexity: $O(E \log V)$

Sorting the Edges: $E \log V$

- The `sorted()` function runs in $O(E \log E)$ time (Linearithmic time)
- The maximum number of edges in a simple graph is V^2 (every vertex is connected to every other vertex, like the complete graph K)
- So we can sub in V^2 to get $O(E \log V^2)$ or $O(V^2 \log V^2)$. And constants are ignored in Big-O, so we can factor out the 2 and remove it
- So we get $O(E \log E) \equiv O(E \log V)$, these are used interchangeably, but V is seen more often

Makeset Operations: $O(V)$

- Initialising the starting sets takes $O(1)$ per vertex, totalling to $O(V)$

Union-Find Operations and Iterating through Edges: $O(E \log V)$

- Kruskal's uses a `for` loop to iterate through E edges
- For each iteration, we call the `find()` function twice (for both vertices (u and v) of the edge) to check for cycles
- A non-optimised `find()` call would take $O(V)$ time (see pre-optimised graph in Path Compression), leading to an overall time complexity of $O(V^2)$
- But the `find()` function optimised with path compression takes $O(\log V)$ time
- So overall, we get $E \times O(\log V) = O(E \log V)$ time

Overall Time Complexity:

- The sorting step dominates, resulting in a total worst-case running time of $O(E \log V)$

Space Complexity: $O(V + E)$

- **Non-Auxiliary:** Storing the given vertices and edges requires $O(V + E)$ space

Auxiliary Space Complexity:

- The `rep` dictionary takes $O(V)$ space
- The `mst` array has $|V - 1|$ edges, resulting in linear space complexity $O(V)$
- The `sorted()` function doesn't sort the list-in place, but creates a new copy of the edges before sorting. Resulting in $O(E)$ auxiliary space
- **Recursion:** When `find()` calls itself, the paused jobs are stored in memory, while path compression reduces this stack, a worst-case scenario would involve a depth of $O(V)$

Overall Space Complexity:

- Both the Auxiliary and Total space complexity is $O(V + E)$

Prim's Algorithm:

_**Note:** Revisit the Minimum Spanning Trees > Generic MST Algorithm Framework

- A "Vertex-First" approach to finding a Minimum Spanning Tree
- Unlike Kruskal's, which builds a forest that eventually merges, Prim's grows a single tree outwards from the starting vertex (Greedy Approach)

Cutting the Graph:

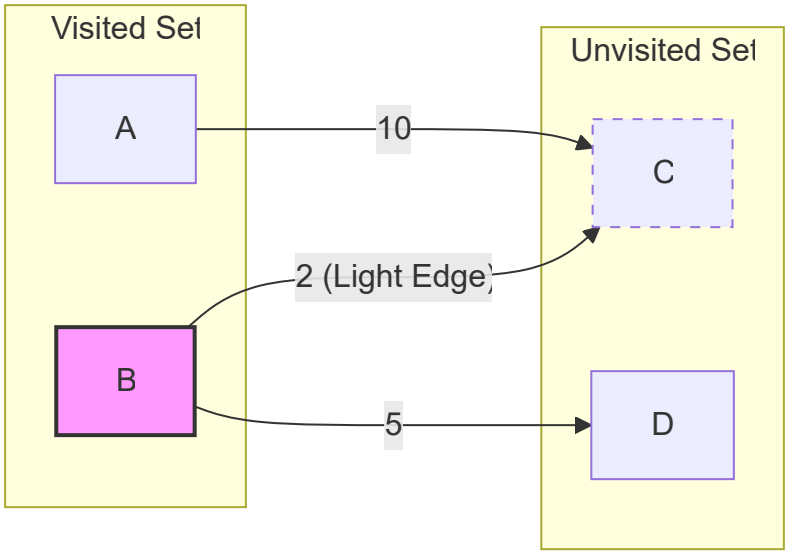
- At every step, the algorithm partitions the graph into two groups:
  1. The **Visited Set** U
  2. The **Unvisited Set** $V \setminus U$ (set V minus set U)

Method:

1. **Set Up:** Pick any starting vertex u to be the first element in set U. The set of MST edges is initially empty $A = \emptyset$
2. **Identify the Cut:** Locate all edges that **cross the cut**, meaning that one endpoint is in U (visited) and the other is in $V \setminus U$ (unvisited)
3. **The Light Edge:** From these crossing edges, identify the **Light Edge** (the one with the minimum weight)
4. **Update:** Add the Light Edge (u, v) to the set A, and move the vertex v into the visited set U
5. **Greedy Loop:** Repeat until $U = V$ (the MST includes all vertices), and return A

Diagram to show a Light Edge:

- The "space" between the visited and unvisited set is the "Cut", all edges that cross the cut are candidates to be the next chosen edge
- **Light Edge:** Out of these edges the shortest one is chosen as the Light Edge



Data Structure: The Priority Queue (Min-heap)

- A complete binary tree that satisfies the **heap property:** for every node v, the value of the parent of node v is less than or equal to the value of the node v, (except when v is the root node)
- This guarantees that picking the root node will always give you the smallest item, making it perfect for **Priority Queues** as we can get the minimum value in $O(1)$

Properties:

- **Completeness:** Every level of the tree is fully filled, except possibly the last level, which is filled from left to right. This keeps the tree balanced
- **Efficiency:** Because the tree is balanced, it is very fast to manage
- **The Min-Heap Property:** $Value(Parent) \leq Value(Child)$

Operation	Time Complexity	Why?
Get Min	$O(1)$	It's always at the root.
Insert	$O(\log n)$	You add at the bottom and "bubble up" to restore the property.
Extract Min	$O(\log n)$	You remove the root, move the last item up, and "bubble down."

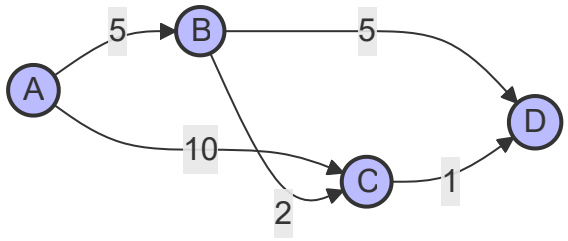
Code Implementation: Prim's Algorithm (Tested in VS Code)

```
import heapq # Import to use a heap queue (a Priority Queue)

class Graph:
    def __init__(self, vertices, adj_list):
        # The list of all vertices in the graph
        self.vertices = vertices
        # The adjacency list format: {node: [(weight, neighbour), ...]}
        self.adj = adj_list

def prim_mst(G, start_node):
    # The list of edges (tuples) that will make up the final MST
    mst_edges = []

    # The set of vertices we have already reached and connected (Set U)
    visited_set = {start_node}

    # A Min-Heap data sructure storing the "Frontier" of available edges
    # Edges are tuples in the format: (weight, u, v)
    frontier_edges = []

    # G.adj[start_node] is the list of adjacent edges
    # Add these edges to the Priority Queue (Min-Heap)
    for weight, v in G.adj[start_node]:
        heapq.heappush(frontier_edges, (weight, start_node, v))

    # The Greedy Loop that runs until we have V-1 edges
    # If frontier_edges is empty, then there are no edges to reach leftover
unvisited nodes (prevents an error for disconnected graphs)
```

```python
    while frontier_edges and len(mst_edges) < len(G.vertices) - 1:

        # 3. The Light Edge: Pop the absolute cheapest edge from the heap
        weight, u, v = heapq.heappop(frontier_edges)

        # 4. Update: If the destination v is NOT in the visited set (it is
unvisited)
        if v not in visited_set:
            # Move vertex v into the visited set U
            visited_set.add(v)
            # Add the chosen edge to our MST
            mst_edges.append((u, v, weight))

            # Identify the new Cut: Add all edges from the newly visited
vertex v
            for next_weight, neighbour in G.adj[v]:
                if neighbour not in visited_set:
                    heapq.heappush(frontier_edges, (next_weight, v,
neighbour))

    return mst_edges

if __name__ == "__main__":
    # Example Graph
    nodes = ['A', 'B', 'C', 'D']

    # Adjacency list format: {node: [(weight, neighbour), ...]}
    adj = {
        'A': [(10, 'C'), (5, 'B')],
        'B': [(5, 'A'), (2, 'C'), (5, 'D')],
        'C': [(10, 'A'), (2, 'B'), (1, 'D')],
        'D': [(5, 'B'), (1, 'C')]
    }

    my_graph = Graph(nodes, adj)
    mst = prim_mst(my_graph, 'A')

    print("Edges in the Minimum Spanning Tree:")
    for u, v, weight in mst:
        print(f"{u} --({weight})-- {v}")
```

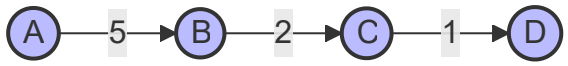
For this Code Example, here is the Original Graph:

Note: *Ignore the arrows, Obsidian won't let me use undirected graphs*



And here is the returned MST:

- The total minimum cost to connect this graph is **8** $(5 + 2 + 1)$



Tracing the Example:

1. Start at A: `visted_set = {A}`

- Edges pushed to the heap: `(5, A, B)` and `(10, A, C)`

2. Pop Light Edge (5, A, B):

- B is unvisited, so we Add to MST, `visited_set = {A, B}`
- Edges pushed from B: `(2, B, C)` and `(5, B, D)`

3. Pop Light Edge (2, B, C):

- C is unvisited, Add to MST, `visited_set = {A, B, C}`
- Edges pushed from C: `(1, C, D)`
- `(10, C, A)` isn't pushed because A is already in the set

4. Pop Light Edge (1, C, D):

- D is unvisited, so we Add to MST, `visited_set = {A, B, C, D}`
- Edges pushed from D: None *(Both of its neighbours, B and C, are already in the `visited_set`, so our optimisation ignores them)*

5. Algorithm Terminates:

- The `visited_set` now contains all 4 vertices ($U = V$), and our MST has exactly 3 edges ($|V| - 1$)
- The `while` loop successfully stops early. The remaining, more expensive edges (like the `(5, B, D)` edge) are simply left behind in the heap, saving us computational time

Final MST Result: `[(5, A, B), (2, B, C), (1, C, D)]`
Total Minimum Cost: 8

Min-Heap Progression:

```
=== Step 1 ===
Min-Heap: [5, 10]
Tree: Root(5) -> Left(10)

=== Step 2 ===
Min-Heap: [2, 10, 5]
Tree: Root(2) -> Left(10), Right(5)

=== Step 3 ===
Min-Heap: [1, 10, 5]
Tree: Root(1) -> Left(10), Right(5)

=== Step 4 & 5 ===
Min-Heap: [5, 10]
Tree: Root(5) -> Left(10)
```

Prim's Algorithm: Complexity Analysis

Note: *Our Implementation could be improved with Fibonacci Heaps and their* `decrease-key` *optimisations*

Time Complexity: $O(E \log V)$

- Every edge connected to a visited vertex is pushed onto the Min-Heap. In the worst-case scenario we push all E edges onto the heap
- Pushing each edge into the heap (by weight) takes $O(\log E)$ time

Popping off the Root: $O(\log E)$

- It takes $O(1)$ time to find the smallest edge, but popping off the root breaks the Min-Heap tree structure
- Because the Min-Heap tree is a **Complete Binary Tree**, its maximum height is always $\log E$ so restructuring the heap takes a maximum of $\log E$ swaps

Overall Time Complexity:

- All these operations combined takes up to $O(E \log E)$ time

---

Space Complexity: $O(V + E)$

Non-Auxiliary Space: $O(V + E)$

- The graph is stored using an Adjacency List, which requires $O(V + E)$ memory to map every vertex to its connected edges

Auxiliary Space: $O(V)$ **or** $O(E)$

- **The Visited Set (U):** Stores a maximum of V vertices, taking $O(V)$ space
- **The MST Array (A):** Stores exactly $V - 1$ edges, taking $O(V)$ space
- **The Priority Queue:** In our standard implementation, the heap can hold up to E edges in the worst case, taking $O(E)$ space

Overall Space Complexity:

- Combining the Adjacency List and the auxiliary data structures results in a total worst-case space complexity of $O(V + E)$

Comparison: Kruskal's vs Prim's

Similarities: Both algorithms are Greedy, and both will successfully find a valid Minimum Spanning Tree with the exact same total minimum weight

Feature	Kruskal's Algorithm	Prim's Algorithm
Growth Strategy	**Edge-First:** Builds a forest of multiple small trees that eventually merge together.	**Vertex-First:** Grows a single, continuous tree outwards from a starting point.
Data Structure	Union-Find (Disjoint Sets / Hash Map)	Priority Queue (Min-Heap)
Sorting	Sorts all edges globally at the very beginning.	Sorts edges dynamically as new vertices are discovered.
Cycle Prevention	Explicitly checks if `find(u) == find(v)`.	Implicitly avoids cycles by only picking unvisited vertices (`v not in U`).
Best Use Case	**Sparse Graphs** (fewer edges). The initial $O(E \log E)$ sorting step is very fast when E is small.	**Dense Graphs** (many edges). The algorithm can ignore thousands of expensive edges without ever processing them.
Standard Time Complexity	$O(E \log V)$	$O(E \log V)$

Related Pages:

- Intro to Graph Theory
- Shortest Paths, Cycles & Connectivity
- Trees & Isomorphism
- Breadth-First Search
- Depth-First Search